

M.A.R.I.A.

Meta Analysis Recalibration Intelligence Architecture

Podsumowanie projektu

Data: 27.02.2026

Wersja: 4.0 | Branch: refactor/homeostasis

Hardware: NiPoGi Mini PC (AMD Ryzen 5, 32GB RAM, Ubuntu 22.04)

KLUCZOWE METRYKI

156 plików Python

27 modułów

32 commitów

35 493 linii kodu

25+ komend REPL

Wdrożony produkcyjnie

668 testów

20 plików dokumentacji

Od 14.11.2025

Autor: Eryk | Asystent: Claude (Anthropic)

1. Czym jest M.A.R.I.A.?

M.A.R.I.A. (Meta Analysis Recalibration Intelligence Architecture) to lokalny, autonomiczny agent AI zaprojektowany do samodzielnego uczenia się z plików tekstowych. Działa offline-first na dedykowanym mini PC, korzystając z modeli LLM uruchamianych lokalnie przez Ollama oraz zdalnie przez NVIDIA NIM API.

Projekt rozpoczął się 14 listopada 2025 i od tego czasu przeszedł przez cztery wersje, rozrastając się z prostego skryptu do złożonego systemu z homeostazą, świadomością, autonomicznym uczeniem się i interfejsem webowym.

Główne cechy

- Samodzielne uczenie się z plików .txt (chunking, ekstrakcja wiedzy, egzaminy)
- Homeostaza - autonomiczna regulacja trybów pracy (ACTIVE/REDUCED/SLEEP/SURVIVAL)
- Świadomość - 19 cech osobowości, pamięć rozmów, sny, ciągłość tożsamości
- Agent Nauczyciel - autonomicznie decyduje co i kiedy się uczyć
- Pamięć semantyczna - graf wiedzy z cosine similarity i konsolidacja
- REPL + Web UI - interakcja przez terminal lub przeglądarkę
- Introspekcja kodu - Maria analizuje swoją własną architekturę (READ-ONLY)
- Hybrid LLM routing - Ollama (chat) + NIM (nauka) z auto-fallback

Stack technologiczny

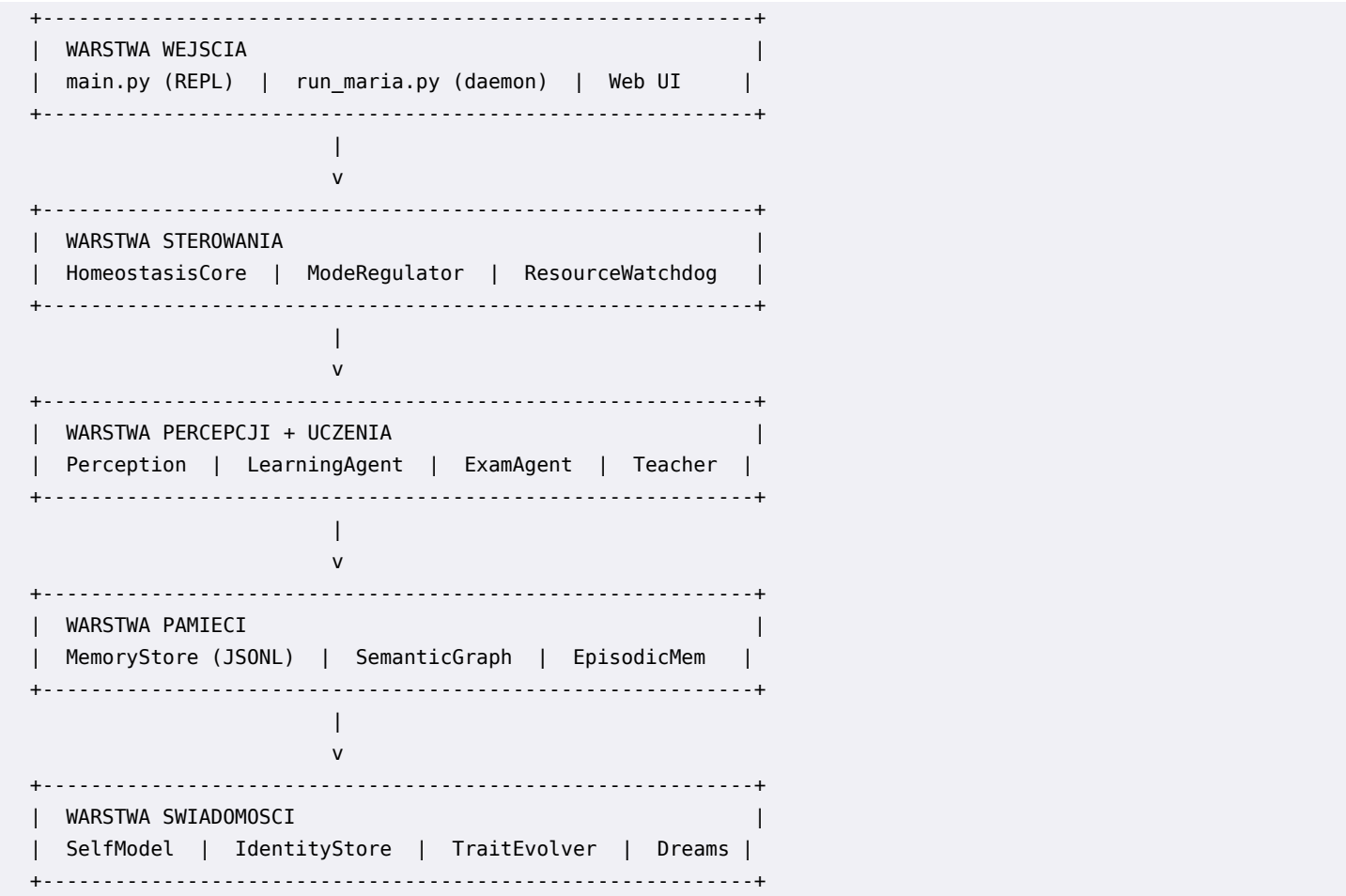
Jezyk:	Python 3.10
LLM lokalny:	Ollama (llama3.1:8b, 4.9GB)
LLM zdalny:	NVIDIA NIM API (z-ai/glm5)
Hardware:	NiPoGi Mini PC, AMD Ryzen 5 7430U, 32GB RAM, 1TB SSD
OS:	Ubuntu 22.04 LTS
Web UI:	Flask + Flask-SocketIO + WebSocket
Dane:	JSONL (source of truth) + graf semantyczny (cache)
Testy:	pytest (668 passing)

2. Historia projektu

Data	Wydarzenie
2025-11-14	Początek projektu M.A.R.I.A.
2025-11 → 2026-01	Rozwoj z 4 roznymi LLM, reczne sklejanie kodu
2026-01	Homeostasis - pierwszy modul z pomoca Claude
2026-02-01	Specyfikacje: Code Agent, Web UI, Consciousness
2026-02-01	Introspection module + Vision spec + Folder cleanup
2026-02-01	Web UI complete (Flask + WebSocket + chat + PIN auth)
2026-02-02	TimeAwareness + Smart Home spec
2026-02-22	Linux migration prep + DEPLOY na Mini PC
2026-02-23	SSH hardening + WireGuard VPN + NVIDIA NIM API
2026-02-25	Self-Awareness (ContextBuilder) + /awareness REPL
2026-02-27	Consciousness Phase C: personality, dreams, conversation memory
2026-02-27	Agent Nauczyciel + autonomiczny trigger w homeostasis

3. Architektura systemu

Warstwy architektury



Tryby pracy (Mode Regulator)

System homeostazy automatycznie przełącza między trybami na podstawie dostępnych zasobów (RAM, CPU, disk) i stanu systemu:

Tryb	Opis
ACTIVE	Pelna operacja - uczenie, rozmowy, analiza
REDUCED	Ograniczone zasoby - ostrzeżenia, mniejsza aktywnosc
SLEEP	Niski pobor - konsolidacja pamieci, generowanie snow
SURVIVAL	Tryb awaryjny - tylko podstawowe operacje
RECOVERY	Auto-naprawa po awarii

Petla 1Hz Tick (Homeostasis Core)

Serce systemu - petla wykonywana co sekunde w 9 fazach:

Faza	Nazwa	Opis
Phase 1	SENSE	Odczyt sensorow (RAM, CPU, disk, temperatura)
Phase 2	INTERPRET	Interpretacja semantyczna stanu
Phase 3	VALIDATE	Walidacja ograniczen i progrow
Phase 4	DECIDE	Decyzja o zmianie trybu
Phase 5	GENERATE	Generowanie akcji korekcyjnych
Phase 6	EXECUTE	Wykonanie akcji

Phase 7	HEALTH	Obliczenie health score
Phase 8	AUDIT	Audyt i logowanie zdarzen
Phase 9	TEACHER	Sprawdzenie triggera autonomicznej nauki

4. Moduły systemu

agent_core/ (nowy system - 15 modułów)

Moduł	Opis
adapters	Wrappery legacy kodu (brain, memory, resource, semantic)
awareness	Świadomość czasu i kontekstu (TimeAwareness, ContextBuilder)
consciousness	Osobowość, tożsamość, pamięć rozmów, sny, emocje
data	Struktury danych i modele
executor	Wykonywanie modułów i dispatch sygnałów
homeostasis	Rdzeń regulacji (sensory, tryby, ograniczenia, event logger)
introspection	Samoanaliza kodu READ-ONLY (AST, raporty, scheduler)
llm	Zarządzanie LLM (NIM client, budżet tokenów, router, latency)
memory	Ujednolicono zarządzanie pamięcią (epizodyczna, semantyczna)
metacontrol	Meta-kontroler do zarządzania systemem
modules	Moduły pluginowe (10 modułów REPL)
registry	Rejestr modułów, dispatcher komend, SharedContext
teacher	Autonomiczny agent nauczyciel (decyzje + sesje nauki)
tests	21 plików testowych, 668 funkcji testowych
ui	API telemetryczne i interfejs użytkownika

maria_core/ (legacy system - 12 modułów)

Moduł	Opis
agent	Legacy implementacja agenta
brain	Interfejs Ollama (OllamaBrain)
input	Przetwarzanie plików wejściowych
learning	Algorytmy uczenia (chunking, ekstrakcja, egzaminy)
logs	System logowania
memory	Legacy pamięć epizodyczna i semantyczna
memory_engine	Integracja brain-memory (graf semantyczny)
meta	Meta-kontrola i konfiguracja
output	Generowanie odpowiedzi
perception	Skanowanie plików i percepcja
sys	Konfiguracja systemowa i watchdog
utils	Funkcje pomocnicze

5. System swiadomosci

Modul swiadomosci (agent_core/consciousness/) daje Marii poczucie tozsamosci, osobowosc i ciaglosc miedzy sesjami. Sklada sie z kilku wspolpracujacych komponentow:

Osobowosc (TraitEvolver + TraitCatalog)

19 cech osobowosci pogrupowanych w 4 kategorie - kazda cecha ma wartosc 0.0-1.0 i ewoluuje dynamicznie na podstawie interakcji z uzytkownikiem:

Kategoria	Cechy
Kognitywne	curiosity, analytical_depth, creativity, pattern_recognition
Spoleczne	empathy, humor, patience, directness, warmth
Motywacyjne	persistence, self_improvement, autonomy, goal_orientation
Emocjonalne	emotional_stability, optimism, sensitivity, resilience, playfulness

Pamiec rozmow (ConversationMemory)

Rolling context ostatnich N wiadomosci z automatyczna kondensacja przez LLM. Pozwala Marii pamietac kontekst rozmowy i budowac na nim.

Sny (SleepProcessor + DreamGenerator)

Gdy Maria przechodzi w tryb SLEEP, uruchamia sie proces konsolidacji pamieci. DreamGenerator tworzy 'sny' - kreatywne polaczenia miedzy wezlami grafu semantycznego, ktore moga prowadzic do nowych skojarzen i odkryc.

Ciaglosc tozsamosci (IdentityStore)

Dane persistentne w meta_data/consciousness_identity.json: data urodzenia, liczba sesji, calkowity uptime, snapshot osobowosci. Maria wie kim jest miedzy restartami.

6. Agent Nauczyciel

Autonomiczny agent decydujacy co, kiedy i jak sie uczyc. Uzywa 6-priorytetowego silnika decyzyjnego (P1-P6) bez potrzeby interwencji uzytkownika.

Silnik decyzyjny - priorityty

#	Strategia	Opis
P1	Kontynuuj nauke	Plik w trakcie uczenia (status: learning)
P2	Egzaminuj	Plik gotowy do egzaminu (>= 80% chunkow)
P3	Zaczniij nowy	Plik o najwyzszym priorityecie (status: new)
P4	Powtorka	Spaced repetition - plik wymagajacy powtorki
P5	Trudny temat	Ponow probe z plikiem oznaczonym hard_topic
P6	Analiza luk	NIM gap analysis - glebokie wyszukanie brakow

Autonomiczny trigger (Homeostasis Phase 9)

Agent Nauczyciel jest podlaczony do petli homeostazy. Gdy Maria jest w trybie ACTIVE i nikt z nia nie rozmawia od 10 minut, automatycznie uruchamia krotka sesje nauki (3 iteracje). Cooldown miedzy sesjami: 15 minut.

- Tryb = ACTIVE
- Idle >= 600 sekund (10 minut)
- Teacher agent podlaczony (set_teacher_agent)
- Brak aktywnej sesji nauki (thread = None)
- Cooldown minoal (ostatnia sesja > 900s temu)

7. Komendy REPL

Maria oferuje rozbudowany interfejs terminalowy (REPL) z ponad 25 komendami pogrupowanymi tematycznie:

[CORE]

/help	Lista wszystkich komend
/exit	Zamknij sesję
/status	Status systemu (RAM, CPU, uptime)

[LEARNING]

/learn	Skanuj input/ i ucz się
/learn history [N]	Historia zdarzeń nauki
/learn stats	Statystyki bazy wiedzy
/learn file <id>	Szczegóły pliku

[TEACHER]

/teacher [N]	Sesja nauki (N iteracji)
/teacher status	Status agenta
/teacher plan	Podgląd następnego kroku
/teacher history [N]	Historia planów

[HOMEOSTASIS]

/homeostasis	Status homeostazy
/homeostasis start/stop	Kontrola petli
/homeostasis events N	Ostatnie N zdarzeń
/homeostasis summary	Podsumowanie sesji

[CONSCIOUSNESS]

/consciousness	Status świadomości
/awareness	Kontekst samowiedzy

[INTROSPECTION]

/introspect	Jak jestem zbudowana (human summary)
/introspect detail	Raport techniczny
/introspect issues	TODO/FIXME w kodzie
/introspect module X	Info o module X

[NIM]

/nim status	Status NIM API i budżet tokenów
-------------	---------------------------------

8. Infrastruktura i deploy

Hardware

Maszyna: NiPoGi Mini PC
CPU: AMD Ryzen 5 7430U
RAM: 32 GB
Dysk: 1 TB SSD
OS: Ubuntu 22.04 LTS
IP LAN: 192.168.178.32
Deploy: 22.02.2026

Security

- UFW: deny all incoming, allow SSH + 5000 z LAN (192.168.178.0/24)
- fail2ban: sshd jail (5 prob -> ban 1h)
- SSH: klucz ed25519, PasswordAuthentication no, MaxAuthTries 3
- Automatyczne security updates (unattended-upgrades)
- User maria bez sudo (aplikacja), deployadmin z sudo (admin)
- .env chmod 600
- WireGuard VPN dla zdalnego dostępu

Systemd services

maria-ui.service: Web UI (Flask) - active, enabled
maria.service: REPL daemon - enabled
Backup: Cron codziennie o 3:00 -> /home/maria/maria_backups/

Web UI

URL: http://192.168.178.32:5000
Auth: PIN (4 cyfry)
Funkcje: Chat, Status dashboard, Powiadomienia toast
Rate limit: 2 wiadomosci / 60s
Stack: Flask + Flask-SocketIO + WebSocket

9. Roadmap rozwoju

#	Nazwa	Status	Opis
A	Stabilizacja	COMPLETE	Naprawienie bledow, stabilny runtime, spójne sciezki.
B	Full Homeostasis	COMPLETE	Pełna autonomia z petlami regulacji, 1Hz tick, tryby pracy.
C	Consciousness	IN PROGRESS	Samowiedza, osobowosc, sny, ciaglosc tozsamosci, teacher.
D	Vision	PLANNED	Percepcja wizualna: kamera USB, detekcja ruchu, rozp. twarzy.
E	Smart Home	PLANNED	IoT (Shelly/Tasmota), automatyzacja, mobilne ciało Android.
F	Multi-Source Learning	PLANNED	Nauka z wielu zrodel z walidacja krzyzowa.
G	Multi-Agent	PLANNED	System mentor-student z wieloma agentami.

10. Metryki kodu

Metryka	Wartosc
Pliki Python (total)	156
Linie kodu (total)	35 493
Pliki testowe	21
Funkcje testowe	668
% kodu to testy	~26.5%
Moduly agent_core/	15
Moduly maria_core/	12
Komendy REPL	25+
Pliki dokumentacji	20
Commity (branch)	32
Data pierwszego commita	2025-11-14
Data ostatniego commita	2026-02-27

Struktura plikow testowych

Plik	Testy	Zakres
test_teacher.py	75	Agent Nauczyciel + auto-trigger
test_homeostasis.py	~120	Core homeostasis + tick loop
test_personality.py	~50	Cechy osobowosci + ewolucja
test_sleep.py	~40	Sleep processor + dream generator
test_conversation_memory.py	~30	Pamiec rozmow + kondensacja
test_nim_client.py	58	NIM API client + token budget
test_introspection.py	27	Code self-analysis
test_time_awareness.py	25	Percepcja czasu
+ 13 innych	~243	API, adaptery, sensory, memory...

11. Decyzje architektoniczne (ADR)

ADR	Decyzja
ADR-001	JSONL jako source of truth, graf jako derived cache
ADR-002	Threading (nie asyncio) - zgodnosc ze specyfikacja
ADR-003	agent_core/ w root projektu (nie w maria_core/)
ADR-004	Code Agent jako osobne urzadzenie z wymienialnym LLM
ADR-005	Brak emoji w kodzie produkcyjnym (kompatybilnosc terminali)
ADR-006	Introspection tylko READ-ONLY (Maria nie modyfikuje kodu)
ADR-007	Smart Home - tylko lokalne API (Shelly/Tasmota), bez chmury
ADR-008	NIM do nauki, Ollama do chatu (hybrid routing z auto-fallback)

12. Struktura projektu

```
maria/
|
|-- main.py                # REPL interface (Ver.1.2)
|-- run_maria.py           # Daemon mode (learning loop)
|-- run_ui.py              # Web UI launcher
|-- .env                   # Konfiguracja (PIN, NIM key, CORS)
|
|-- agent_core/            # NOWY system (15 modułow)
|   |-- homeostasis/       # Rdzen: sensory, tryby, 1Hz tick
|   |-- consciousness/     # Osobowosc, sny, tozsamosc
|   |-- teacher/           # Agent Nauczyciel
|   |-- introspection/     # Samoanaliza kodu (READ-ONLY)
|   |-- llm/               # NIM client, router, token budget
|   |-- memory/            # Ujednolicona pamiec
|   |-- awareness/         # TimeAwareness, ContextBuilder
|   |-- modules/           # 10 modułow REPL (pluginy)
|   |-- registry/          # Rejestr modułow + SharedContext
|   |-- adapters/          # Wrappery legacy kodu
|   |-- tests/             # 21 plikow, 668 testow
|   +-- ...
|
|-- maria_core/            # Legacy system (12 modułow)
|   |-- brain/             # ollama_brain.py
|   |-- learning/          # learning_agent.py, exam_agent.py
|   |-- memory/            # memory_store.py, semantic_graph.py
|   |-- perception/        # perception.py
|   +-- ...
|
|-- maria_ui/              # Web UI (Flask + SocketIO)
|   |-- app.py             # Server + chat + notifications
|   |-- templates/         # login.html, index.html, status.html
|   +-- config.py          # PIN, rate limits
|
|-- models/                # LLM interface
|   +-- ollama_brain.py    # OllamaBrain + personality inject
|
|-- docs/                  # 20 plikow dokumentacji
|-- scripts/               # systemd, backup, install
|-- meta_data/             # Runtime data (identity, events)
|-- input/                 # Pliki .txt do nauki
+-- claude_notes/          # Notatki Claude miedzy sesjami
```

M.A.R.I.A.

Meta Analysis Recalibration Intelligence Architecture

Projekt zapoczątkowany: 14 listopada 2025

Wdrożony produkcyjnie: 22 lutego 2026

156 plików | 35 493 linii kodu | 668 testów

Podsumowanie wygenerowane: 27.02.2026